IVEY | Publishing

W38454

TECHNICAL NOTE

DATA MODELLING WITH BARKER NOTATION

SYNOPSIS

Entity-Relationship (ER) data modelling is a critical technique used by data experts to help design new systems, utilize existing systems, and replace old systems. It is grounded in a theoretical notion called *data abstraction*, which simply means separating data details from implementation details. In other words, data modelling offers a way to simplify things and focus on "just the data."

The objectives of data modelling are to conceptualize, normalize, standardize, communicate, and accurately represent a real-world domain. In practical terms, the result of the data modelling process is a schema: a graphical diagram. Creating data model schemas can seem a little complicated at first, mainly because it involves learning precise rules for eliminating complexity. However, the rules are actually quite straightforward, and once you begin to apply them to real scenarios, you will be creating high-quality data models in no time.

The remainder of this note describes the different levels of data modelling, as well as its objectives and benefits, provides a brief history of graphical modelling approaches, and focuses on a particular graphical rule set for creating logical data models called *Barker notation*.

LEVELS OF MODELLING

ER modelling occurs on three different levels:

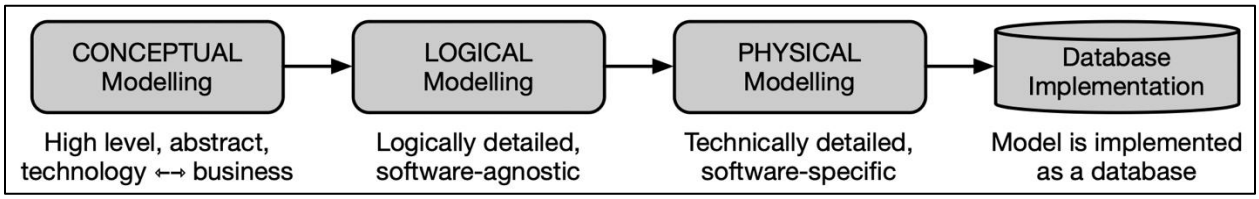


Figure 1. Adapted from Andy Morris, *Data Modeling Explained: Types and Benefits*, accessed April 1, 2024, https://www.netsuite.com/portal/resource/articles/data-warehouse/data-modeling.shtml.

- **Conceptual data modelling** establishes a high-level abstraction of a domain's information structure, focusing on the identification of key entities and their interrelationships. A conceptual model does not include attributes or storage methods. It is aimed at bridging understanding between business stakeholders and technical teams.
- **Logical data modelling** converts a conceptual model into a detailed blueprint, specifying entities, attributes, relationships, and key identifiers. A logical data model is software agnostic—i.e., it can be implemented on any relational database management system (DBMS), whether one from Oracle or Microsoft or an open-source application.
- **Physical data modelling** involves step-by-step implementation of a logical model using a specific DBMS, detailing tables, columns, data types, primary keys, foreign keys (FKs), constraints, and indexes, along with physical storage considerations such as partitioning and clustering, to optimize performance and integrity.

OBJECTIVES OF MODELLING

Learning to capture real-life situations in a data model will help you develop the following skills:

a. **Conceptualization.** The data modelling process begins with examining a real-life situation and conceptualizing the relevant *entities*, or objects and notions of interest. Through successive iterations, increasingly detailed data elements are identified, including *attributes* (properties of entities), *identifiers* (unique attributes of entities), and *relationships* (associations between entities). By abstracting and conceptualizing data separately from software and physical storage issues, the modeller creates a holistic, integrated, "clean" view of data without distractions.

b. **Normalization.** Database normalization is a critical process aimed at organizing data to minimize *redundancy* (data duplication) and enhance *integrity* (accuracy and reliability). Data modelling naturally fosters normalization. That is, by carefully applying the rules for defining entities, attributes, identifiers, and relationships in the data model, initial normalization will be achieved, along with the ability to better manage data dependencies, constraints, scalability, security, and more.

c. **Standardization.** Data modelling entails a standardized approach to documenting the data architecture of a system, which is crucial for its maintenance, modification, and scalability. The data model documentation acts as a vital reference for both current and future development efforts, promoting consistency and clarity throughout the system's lifecycle. Moreover, standardization ensures adherence to uniform data formats, naming conventions, and modelling methodologies, thus enhancing interoperability and maintaining high data quality across diverse system components.

d. **Communication.** The visual nature of data models enhances stakeholder communication in database design and development, bridging expertise among systems analysts, designers, developers, and end-users. Although they may initially seem complex, with basic training, these models can serve as essential communication tools, providing clear, intuitive representations of data structures and relationships. This facilitates discussions, systems design validation, and effective data analysis and reporting. Accurately defined data structures support the design of queries and reports that reflect real-life dependencies, thus enhancing business intelligence and decision-making.

e. **Representation.** The overarching goal of a data model is to represent important data aspects of a real-world domain. While technical correctness can be achieved by following the rules, there is also a significant "art" to data modelling that demands not only technical expertise from the modeller but also domain-specific details and information—specialized knowledge that is often known only to C-level executives or other insider experts. This knowledge must be carefully and thoughtfully drawn out and interpreted. As a result of this artistic process of interpretation, there may be (and often are) multiple valid ways to solve any given data management problem.

When designing data models, legal and ethical considerations regarding data integrity, privacy, and security should be considered. Database experts must ensure that sensitive information is protected, access controls are implemented, and data accuracy is maintained. For example, in handling data related to health care that are considered protected health information (PHI), the Health Insurance Portability and Accountability Act (HIPAA) must be complied with. In handling data involving financial transactions, the Payment Card Industry Data Security Standard (PCI DSS) requirements must be followed. Online collection of data from European citizens must conform to the European General Data Protection Regulation (GDPR). Proper data management governance helps safeguard the interests of all stakeholders—customers, employees, companies, shareholders, and everyone else.

Data modelling provides a valuable skillset and "technical language" for conceiving, normalizing, standardizing, communicating, and representing data. Anyone can create or begin using a database—but it takes expert knowledge, skills, and abilities to build and maintain robust, efficient, scalable, *high-quality* data management systems. Learning how to create data models will take you down this path.

A BRIEF HISTORY OF DATA MODELLING

The relational database management system (RDBMS) model, a cornerstone of database technology that is ubiquitous today, was first introduced in 1970 by Edgar F. Codd,[1] a British computer scientist who worked at IBM Corporation. Codd's paper envisioned a new method for storing and retrieving data using a table-based column/row format. Storing data in standardized, interrelated tables would allow them to be instantly reorganized for a variety of different purposes without needing to be restructured for every different use or program. The relational database model contrasted sharply with the then-prevalent hierarchical and network database models by offering a more flexible and intuitive way to represent data and their interrelationships. Codd's relational model laid the conceptual foundation for RDBMS, which used mathematical set theory and predicate logic to reduce data redundancy and improve data integrity.

Following the introduction of Codd's relational model concept, IBM pursued a noncommercial software project in the early 1970s called System R (not to be confused with the statistical software package R). IBM's System R software introduced many of the core concepts from Codd's work into practice, including structured query language (SQL), and heavily influenced the development of subsequent RDBMSs.

During this time, Peter Chen, a computer scientist and professor at Carnegie Mellon University, was exploring methods to visually represent data entities and relationships, resulting in a seminal paper published in 1976.[2] Chen's ER modelling approach, which became known as Chen notation, offered a bridge between the theoretical underpinnings of Codd's relational model and the practical design of RDBMS schemas. It was a method that allowed designers to conceptualize, structure, and visualize database systems before committing to implementation.

Over the following years, other data modelling approaches with varying feature sets began to appear. The integration definition for information modelling (IDEF1X) methodology emerged in the late 1970s as a joint project of the US Air Force and Hughes Aircraft Company. IDEF1X was an elaboration of Chen notation designed for use in very large and highly complex manufacturing contexts, and in 1993, the US National Institute of Standards and Technology (NIST) adopted it as a national standard.

[1] Edgar F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM* 13, no. 6 (1970): 377–387.
[2] Peter P. S. Chen, "The Entity-Relationship Model—Toward a Unified View of Data," *ACM Transactions on Database Systems* 1 no. 1 (1976): 9–36.

While IDEF1X provided a precise and increasingly detailed approach for modelling highly complex systems, data and software engineers who were working in the field on everyday organizational projects needed a faster and simpler method for jotting out elements of a database system. Approaches like IDEF1X were overkill. What emerged was a community convention that became known colloquially as "crow's foot" notation—named due to its usage of the crow's foot symbol (⋞) to visualize relationships between entities.

Then, in the early 1980s, a small group of data engineers at the British technology consultancy CACI decided to develop a more standardized graphical modelling methodology. Led by systems consultant Richard Barker,[3] their objective was to keep the method simple and clear, like crow's foot notation, but with a generalizable and concrete methodology and rule set, so it could be applied consistently from one context to another. Soon after, Barker joined Oracle Corporation, a global database software company, where "Barker notation" became widely adopted and is still in use. Barker provided a detailed explanation of this notation in his 1990 book, *CASE Method*.[4]

By the mid-1990s, additional graphical approaches had appeared. Three such approaches were introduced by three software engineers: the Booch method by Grady Booch, the object modelling technique or OMT by James Rumbaugh, and Object-Oriented Software Engineering or OOSE by Ivar Jacobson. Eventually, these engineers decided to collaborate to build a comprehensive modelling approach they called the unified modelling language (UML), which became popular and widely adopted in support of complex software development projects. However, like IDEF1X, it was quite elaborate and thus not ideal for everyday data modelling tasks in practice.

Among these approaches, Barker notation was considered superior for many business contexts. It was a simpler notation to understand and use than Chen, IDEF1X, or UML, yet more rigorous and standardized than crow's foot notation. The following table summarizes the differences between these data modelling methodologies and highlights the relative advantages of Barker notation.

Table 1: Data Modelling Methodologies

	Pros	**Cons**	**Advantages of Barker**
Chen Notation	Intuitive conceptual representation. Clear distinction between entities and relationships. In use at IBM.	Can become complex with large databases; lacks some modern features such as inheritance.	Barker notation is simpler and more accessible for large-scale projects.
Integration Definition for Information Modelling (IDEF1X)	Strong emphasis on normalization. Precise standards for database design. In use at US DoD and NASA.	Notation can be complex; less intuitive for beginners.	Barker notation offers a more intuitive approach, making it easier for newcomers.
Crow's Foot Notation	Visually intuitive representation of relationships and cardinalities. Widely adopted. In use across various industries.	May not explicitly represent all aspects of data constraints and behaviours; lacks rigour of published standards.	Barker notation provides a similarly intuitive approach but with better consistency via standardized rules.

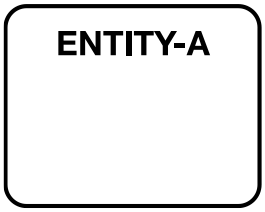
[3] *Barker's Notation*, Wikipedia, accessed April 1, 2024, https://en.wikipedia.org/wiki/Barker%27s_notation.
[4] Richard Barker, *CASE Method: Entity Relationship Modelling* (Wokingham: Addison-Wesley Longman, 1990).

Unified Modelling Language (UML)	Supports a wide range of software development aspects beyond databases. Highly versatile. In use at Microsoft and Google.	Complexity can be overwhelming for pure database design uses; steep learning curve.	Barker notation is more streamlined, intuitive, faster, and easier to apply.
Barker Notation	Simplified, clear, and easy to understand. Effective for communicating database designs. In use at Oracle and SAP.	Lacks some of the advanced (software engineering) features of UML.	Barker notation is recognized for its trade-off between simplicity and rigour.

BASIC BARKER NOTATION

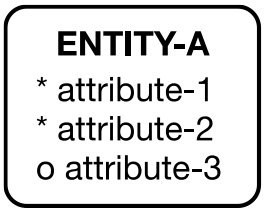
A Barker data model has four main components, each with its own set of design rules.[5]

a. **ENTITIES.** Entities are the building blocks of a data model. Each entity represents a key object from the real world (e.g., a person, place, event, item, idea, etc.) that the database needs to track. Examples of entities in a sales management system might include **CUSTOMER**, **SALESPERSON**, **PRODUCT**, and **INVOICE**. Deciding which entities to include in a data model depends on the domain of interest and the specific requirements of a project or application.



Rules:
- An entity represents a real-world object or concept
- Each entity is drawn as a rectangle with rounded corners
- The entity name is centred and shown in ALL CAPS
- The entity name is always written in the singular form, not plural
- The terminology used in the model should be familiar to users (imposing unfamiliar or "developer" language should be avoided)

b. **ATTRIBUTES.** Attributes are the relevant properties of an entity. Each attribute is an important detail about the entity that we need to track. For instance, the entity **CUSTOMER** might have attributes such as **lastname**, **firstname**, and **phonenum**. As with entities, deciding which attributes to include depends on the domain and requirements.

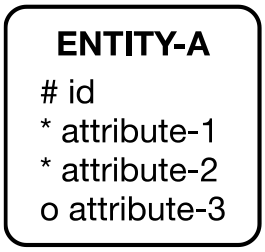


Rules:
- Attributes are listed underneath the entity name
- Attribute names are written in lower case
- Each attribute represents one characteristic of the entity
- An * (asterisk) is used to denote that the attribute is required
- An o (the letter "o") is used to denote that the attribute is optional

c. **IDENTIFIERS**. In relational databases, every instance of an entity must have its own unique identifier. For example, whenever a university or college admits a new student, that student is assigned their own unique identifier (e.g., so that we can keep track of what courses each student has taken, the grades they have achieved, and the degrees they have earned). In most cases, we will create an attribute, for example

[5] This introduction to Barker notation design rules was developed with reference to Frankie Inguanez, *Entity Relationship Modelling: A Short Guide to Designing Entity Relationship Models Using Barker's Notation* (2012).

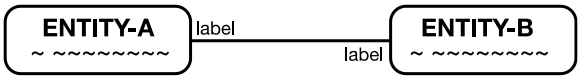
named **id**, to automatically hold the identifier value (e.g., id = 1 is student 1, id = 2 is student 2, and so on). When two or more people enroll who have other attributes that are identical (first and last name of "Chris Jones"), they will remain distinguishable because their **id**'s are unique. Occasionally, an entity will already contain a natural attribute that can serve as the identifier—e.g., a car rental company that captures the **driverslicense** for each of its customers might use that as the identifier. In other situations, multiple attributes are set together to form a composite identifier (more on this later).



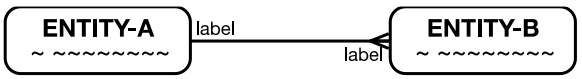
Rules:
- Identifiers are attribute(s) used to uniquely identify each entity
- Identifiers are typically denoted with a leading # (hashtag)
- Every entity must have an identifier
- Identifiers are typically listed as the first attribute(s)
- Multiple attributes can be set to form a "composite identifier"

d. **RELATIONSHIPS**. Relationships between entities can have three primary forms, or *cardinalities*, represented using lines and shapes. Note that every line must be labelled, with a verb, on both ends of the relationship.

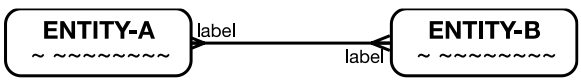
- **One-to-One (1:1)**. A 1:1 relationship indicates that for every instance of **ENTITY-A**, there is one instance of **ENTITY-B**, and for every instance of **ENTITY-B**, there is one instance of **ENTITY-A**. It is drawn as a line between the entities. The 1:1 relationship is relatively rare. Often, data in a 1:1 relationship can be simply stored in a single table without causing redundancy or violating normalization rules. Occasionally however, data is best stored in a 1:1 relationship; for example, sensitive data may need to be separated and held in a different entity to improve security.
  Rule: A 1:1 relationship is drawn using a line between entities



- **One-to-Many (1:m)**. A 1:m relationship indicates that for every instance of **ENTITY-A**, there must be at least one and potentially many instances of **ENTITY-B**, and for every instance of **ENTITY-B**, there must be one instance of **ENTITY-A**. It is drawn as a line with a crow's foot symbol ⋞ on the "many" end. The 1:m relationship is very common.
  Rule: A 1:m relationship includes a crow's foot going into the "many" entity



- **Many-to-Many (m:m)**. A m:m relationship indicates that for every instance of **ENTITY-A**, there must be at least one and potentially many instances of **ENTITY-B**, and for every instance of **ENTITY-B**, there must be at least one and potentially many instances of **ENTITY-A**. While intuitively reasonable, the following example is **NOT the correct way** to model a m:m relationship:



Instead, whenever an m:m relationship occurs, an additional *intersection* entity must be created between the two parent entities, with a 1:m between **ENTITY-A** and **ENTITY-AB**, a 1:m between **ENTITY-B** and **ENTITY-AB**, and both crow's feet entering the intersection **ENTITY-AB**. The

intersection entity may be used to hold additional details specific to this relationship. The m:m relationship is very common.
Rule:
○ A m:m relationship always requires an additional "intersection entity," with both crow's feet pointing into the intersection entity
○ If the intersection entity does not have an obvious corollary from the real world, a common practice is to make up a new name using a combination of the parent-entity names

- **Primary and Foreign Keys**. This note is focused on conceptual and logical data modelling. After a model has been designed, it will be implemented in a relational database management system (RDBMS), at which point unique identifiers in the model will be converted to primary key (PK) fields, and foreign key (FK) fields will be created in related entities. Those already familiar with implementing databases but new to data modelling may be tempted to include FK identifiers as attributes in the data model itself, by writing them inside the entity. This is not the correct treatment. The crow's foot notation is a sufficient indicator of the nature (cardinality) of each relationship. To keep the model as efficient/nonredundant as possible, FKs are not shown in the logical model. All necessary FKs will be generated at implementation time, when the logical model is converted into a physical model.
Rule:
○ FK identifiers are not shown as attributes because they are already indicated using crow's feet

CASE: THE BROKEN PHONE WIZARD PART 1

The Broken Phone Wizard ("The Wizard"), a service dedicated to repairing broken screens on mobile devices, requires an efficient data model to manage its operations. A preliminary meeting with the company's CEO, Jesse Stone, has revealed or hinted at the following data details:

The entities involved include the following:

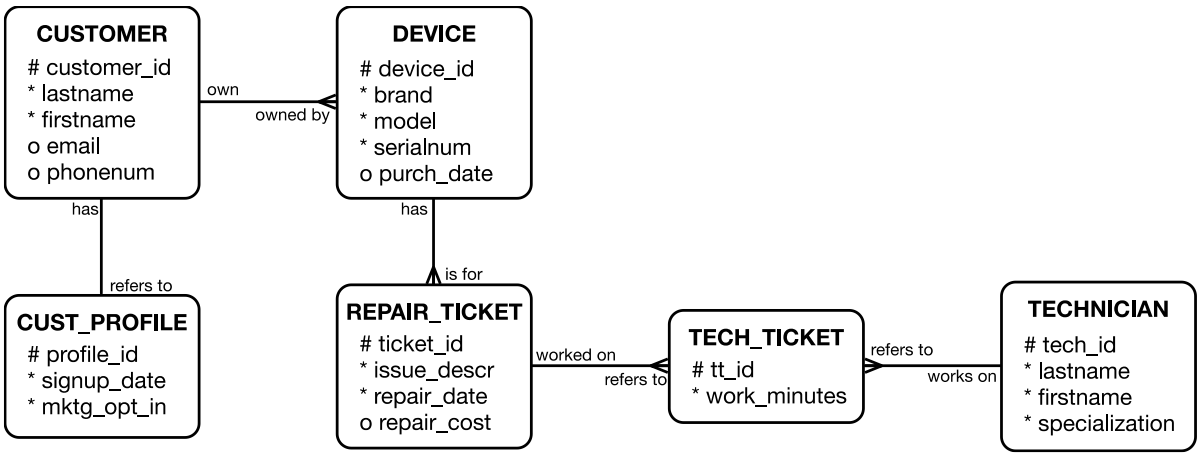
- CUSTOMER, identified by a customer_id, has the attributes lastname, firstname (both required), and optional attributes email and phonenum.
- DEVICE, identified by a device_id, has the required attributes brand, model, and serialnum, with an optional purch_date.
- REPAIR_TICKET, identified by a ticket_id, includes the required attributes issue_descr and repair_date, as well as an optional repair_cost.
- TECHNICIAN, identified by a tech_id, has the required attributes lastname and firstname and an optional attribute specialization.
- CUST_PROFILE, identified by a profile_id, includes the required attributes signup_date and mktg_opt_in.

The relationships between entities involved include the following:

- A 1:m relationship exists between CUSTOMER and DEVICE, as each customer can own multiple devices, but each device is owned by just one customer.
- Another 1:m relationship exists between DEVICE and REPAIR_TICKET, since each repair ticket is for just one device, but a device can have multiple repair tickets.

- A m:m relationship exists between TECHNICIAN and REPAIR_TICKET, since each technician works on many tickets, and each ticket can be worked on by multiple technicians.
- This m:m relationship requires an additional intersection entity that we will call TECH_TICKET, identified by tt_id, with the required attribute work_minutes.
- A 1:1 relationship exists between CUSTOMER and CUST_PROFILE, such that each customer has just one profile, and each profile refers to just one customer.
- Note: TECHNICIANs work on DEVICEs, yet there is no line connecting these entities, as the relationship already exists through REPAIR_TICKET. Drawing another line would be redundant.

Referring to these notes, you can sketch out the following preliminary data model for The Wizard:



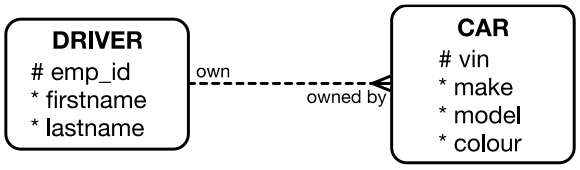
ADVANCED BARKER NOTATION

In addition to the main components of a data model described above, there are a few other concepts and special purpose design rules that are used to model real-world situations.

a. **OPTIONALITY**. The solid lines used to connect the entities above denote mandatory relationships. However, relationships are often a mixture of mandatory and optional, denoted in the model by using dotted relationship lines.
   - **Optional 1:1**. For every instance of **ENTITY-A**, there may exist zero or one instance of **ENTITY-B**, and for every instance of **ENTITY-B**, there may exist zero or one instance of **ENTITY-A**. This relationship type is valid but relatively rare. (NB: when implementing this relationship, an optional foreign key is placed in the entity that is more subjugated or subordinate to the other entity, which may require a judgment call.)
     Scenario: A **DRIVER** may be assigned zero or one parking **SPOT** at work, and a **SPOT** may be assigned to zero or one **DRIVER**.

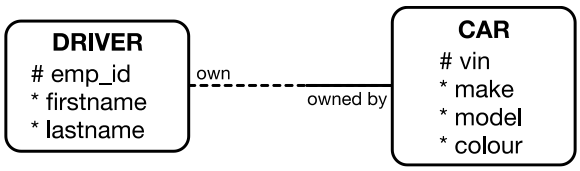
   - **Optional 1:m**. For every instance of **ENTITY-A**, there may exist zero, one, or multiple instances of **ENTITY-B**, and for every instance of **ENTITY-B**, there may exist zero or one instance of **ENTITY-A**.

This relationship type is valid and common. (NB: when implementing this relationship, an optional foreign key is placed in the "many" entity.)
Scenario: A **DRIVER** may own zero, one, or many **CAR**s, while a **CAR** may or may not yet be owned by a **DRIVER**.

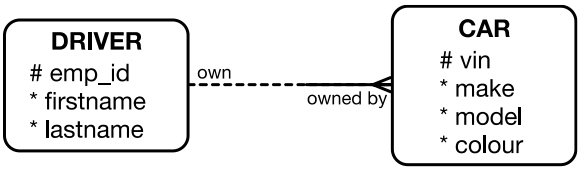


Barker notation also allows the modeller to represent situations where half of a relationship is optional.

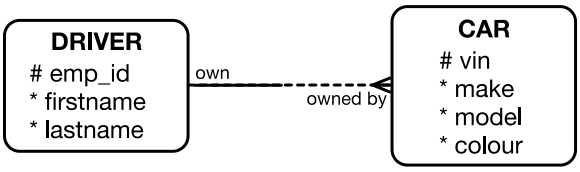
- **Half-Optional 1:1**. For every instance of **ENTITY-A**, there may exist zero or one instance of **ENTITY-B**, but for every instance of **ENTITY-B**, there must exist exactly one instance of **ENTITY-A**. This relationship type is valid but rare. (NB: when implementing this relationship, a mandatory foreign key is placed on the mandatory side.)
  Scenario: A **DRIVER** may own zero or one **CAR**, while a **CAR** must be owned by one **DRIVER**.



- **Half-Optional 1:m—Version 1**. For every instance of **ENTITY-A**, there may exist zero, one, or multiple instances of **ENTITY-B**, but for every instance of **ENTITY-B**, there must exist exactly one instance of **ENTITY-A**. This relationship type is valid and common. (NB: when implementing this relationship, a mandatory foreign key is placed on the mandatory "many" side.)
  Scenario: A **DRIVER** may own zero, one, or many **CAR**s, while each **CAR** must be owned by one **DRIVER**.

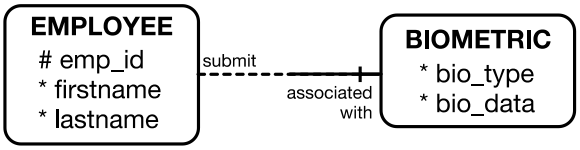


- **Half-Optional 1:m—Version 2**. For every instance of **ENTITY-A**, there must exist one or multiple instances of **ENTITY-B**, but for every instance of **ENTITY-B**, there may exist zero or one instance of **ENTITY-A**. This relationship type is valid but rare. (NB: when implementing this relationship, an optional foreign key is placed on the optional "many" side. However, this will not guarantee that every instance of **ENTITY-A** has a mandatory relationship to an instance of **ENTITY-B**. To address this issue, either a check constraint or specialized application code is required to ensure that every instance of **ENTITY-A** always has at least one related instance of **ENTITY-B.**)
  Scenario: A **DRIVER** must own at least one **CAR**, while a **CAR** may exist but not yet be owned by any **DRIVER**.

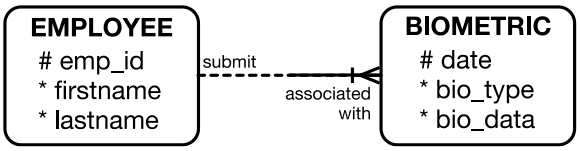


b. **WEAK ENTITIES**. Entities can be either *strong* or *weak*. A strong entity is self-sufficient and usually has a real-world corollary, whereas a weak entity is dependent on one or more strong entities for identification and may not directly refer to anything in the real world. To model a weak entity's reliance on a strong entity in a data model, a unique identifier (UID) bar is added to the relationship line.

- **One-to-One (1:1) Weak Entity**. **ENTITY-A**, a strong entity, is related 1:1 with **ENTITY-B**, a weak entity. Each instance of **ENTITY-A** may be associated with zero or one instance of **ENTITY-B**, while **ENTITY-B** is fully identified with **ENTITY-A**'s identifier attribute, as depicted by a UID bar into **ENTITY-B**, with no additional identifiers required. This relationship type is valid but rare. (NB: when implementing this relationship, a mandatory foreign key is placed on the mandatory side with the UID bar.)
  Scenario: An **EMPLOYEE** (strong entity with unique id) may submit zero or one corresponding **BIOMETRIC**s (weak entity lacking a unique identifier and reliant on the person identifier, per the UID bar), while each **BIOMETRIC** is uniquely identified by **emp_id** and associated with exactly one **EMPLOYEE**.

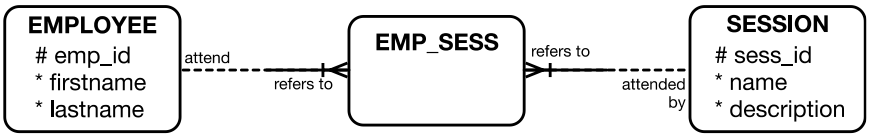


- **One-to-Many (1:m) Weak Entity with Partial Identifier**. **ENTITY-A**, a strong entity, is related 1:m with **ENTITY-B**, a weak entity. **ENTITY-B** is identified with **ENTITY-A**'s identifier attribute, plus any additional partial identifiers. This relationship type is valid and relatively common. (NB: when implementing this relationship, a mandatory foreign key is placed on the mandatory "many" side with the UID bar.)
  Scenario: An **EMPLOYEE** (strong entity with unique id) may submit zero, one, or multiple **BIOMETRIC**s (weak entity with partial identifier, **date**, plus **emp_id** identifier per the UID bar), while each **BIOMETRIC** is associated with exactly one **EMPLOYEE**.



- **Many-to-Many (m:m) Weak Entity—Version 1**. **ENTITY-A**, a strong entity, is related m:m with **ENTITY-B**, also a strong entity. The intersection **ENTITY-AB** happens to be a weak entity that relies on both **ENTITY-A** and **ENTITY-B** for its identification and has no additional attributes. This relationship type is valid and relatively common. (NB: when implementing this relationship, mandatory FKs are placed in the intersection entity.)
  Scenario: An **EMPLOYEE** can attend zero, one, or multiple training **SESSION**s, while a training session can be attended by zero, one, or multiple people. The intersection entity (**EMP-SESS**) is weak (i.e., it lacks any unique identifiers of its own, so it relies on identifiers from **EMPLOYEE** and

SESSION). Note that every combination of **emp_id** + **sess_id** in the **EMP_SESS** entity must be unique, since a given employee can only appear one time for one particular session.

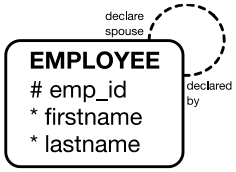


- **Many-to-Many (m:m) Weak Entity—Version 2**. **ENTITY-A**, a strong entity, is related m:m with **ENTITY-B**, also a strong entity. The intersection **ENTITY-AB** is a weak entity whose identification relies on **ENTITY-A** plus an internal attribute from the intersection entity. This relationship type is valid and relatively common. (NB: when implementing this relationship, a mandatory foreign key is placed in the intersection entity.)
- **Scenario**: An **EMPLOYEE** can attend zero, one, or multiple training **SESSION**s, while a **SESSION** can be attended by zero, one, or multiple **EMPLOYEE**s. Attendance is recorded in the intersection entity called **EMP-SESS**, which is a weak entity identified by the **EMPLOYEE** identifier **emp_id** plus an internal attribute **date**. Note that every combination of **emp_id** + **date** in the **EMP_SESS** entity must be unique—in other words, a given employee may only attend a given session once on a particular date, but they *could* attend the same session on multiple different dates.

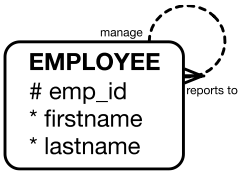
- **When to Use a UID Bar?** To reiterate, a UID bar is typically used in a data model when a weak dependent entity relies on one or more strong entities for its identification. Unlike strong entities, which already have their own unique identifiers, a weak entity does not have an independent unique attribute and instead depends on the identifiers of the related strong entities. The UID bar is placed on the relationship line leading to the weak entity, indicating that the unique identifier of the weak entity is a composite key derived from the identifier(s) of the strong entity(ies) to which it is related. This is crucial for ensuring that each instance of the weak entity is uniquely identifiable within the context of its associated strong entities. When a dependent entity does have a unique identifier, or when a unique identifier can be justifiably created, a UID bar is *not* used.

c. **RECURSION**. Instances of an entity are sometimes related to other instances of the same entity, such as when an employee manages other employees. These situations are modelled as *recursive* relationships.
   - **One-to-One (1:1) Recursive Relationship**. In a 1:1 recursive relationship, each instance of an entity may be associated with at most one other instance of the same entity. (NB: when implementing this relationship, an optional foreign key is created in the recursive entity.)

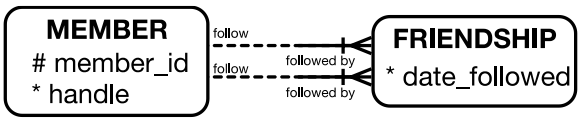
Scenario: Each **EMPLOYEE** may (optionally) declare one other **EMPLOYEE** as their spouse.



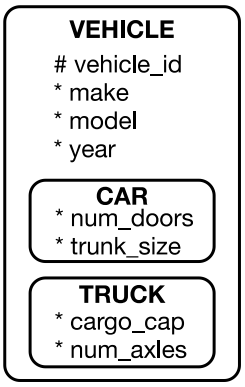
- **One-to-Many (1:m) Recursive Relationship**. In a 1:m recursive relationship, each instance of an entity may be associated with multiple other instances of the same entity, while every subordinate instance may be associated with only one independent instance. (NB: when implementing this relationship, an optional foreign key is created in the recursive entity.)
  Scenario: Each **EMPLOYEE** may (optionally) manage zero, one, or multiple other **EMPLOYEE**s, while each **EMPLOYEE** (optionally) reports back to one manager.



- **Many-to-Many (m:m) Recursive Relationship**. In a m:m recursive relationship, each instance of an entity can be associated with multiple other instances of the same entity; and each of the subordinate instances can be associated with any number of other instances of the same entity. (NB: when implementing this relationship, as with any other m:m relationship, an intersection entity must be created to hold the FKs.)
  Scenario: A social network in which a **MEMBER** may have zero, one, or many **FRIENDSHIP**s with any number of other **MEMBER**s.

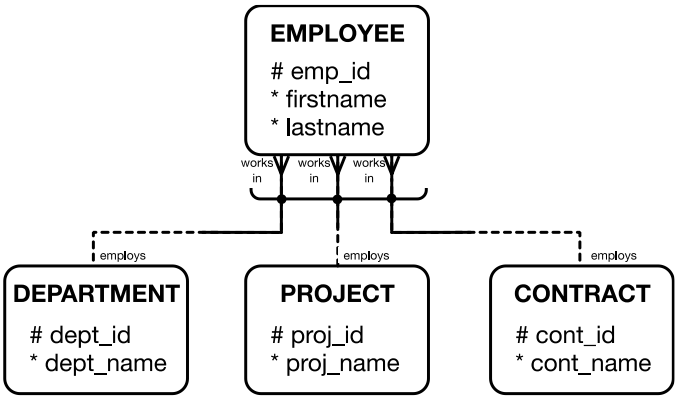


d. **SUPER-ENTITIES**. A super-entity is a specialized entity that holds one or more subentities. The super-entity can have attributes that apply to each sub-entity (i.e., the sub-entities inherit these attributes), while the sub-entities can also have additional attributes that are specific only to that sub-entity. Super-entities allow for further modelling precision of real-world concepts by capturing both shared and distinct characteristics within a single framework.
  Scenario: An organization manages a fleet of different types of **VEHICLE**s, including **CAR**s and **TRUCK**s. This is modelled as a super-entity with common attributes (make, model, year) and subentities with specific attributes for cars (number of doors, trunk size) and trucks (cargo capacity, number of axles). Note that relationship lines may be drawn to both the **VEHICLE** super-entity and to the **CAR** and **TRUCK** subentities, depending on the reality of the situation. (NB: implementing this scenario involves creating a **VEHICLE** table with the common attributes and primary key, as well as **CAR** and **TRUCK** tables with their specific attributes and FKs referencing the **VEHICLE** table's primary key to establish a relationship.)



e. **EXCLUSIVE RELATIONSHIP ARC**. Exclusive relationship arcs in Barker notation represent situations in which an entity can be associated with one and only one of several possible related entities. It is drawn as an arc shape that runs through the relationship options, with a dot indicating which relationships belong to the exclusive list.

Scenario: An **EMPLOYEE** must work in one of a **DEPARTMENT**, a **PROJECT**, or a **CONTRACT**, but not in more than one. Each **DEPARTMENT**, **PROJECT**, and **CONTRACT** employs multiple **EMPLOYEE**s. (NB: implementing this scenario involves creating **EMPLOYEE**, **DEPARTMENT**, **PROJECT**, and **CONTRACT** tables, adding optional FKs in the **EMPLOYEE** table, and enforcing a check constraint, or application logic, to ensure that only one foreign key is populated for each employee to maintain the exclusive relationship).



CASE: THE BROKEN PHONE WIZARD PART 2

From subsequent meetings with staff at The Wizard, you have collected the following additional details:

- Several **optional** relationships exist:
  - The relationship between **CUSTOMER** and **CUST_PROFILE** is optional on the **CUSTOMER** side, but mandatory on the **CUST_PROFILE** side (i.e., a customer might optionally have one associated profile, but every profile must have an associated customer).
  - Each of the remaining 1:m relationships in this model are optional on the 1-side, but mandatory on the m-side (e.g., a customer may optionally have one or more associated devices, while each device must have an associated customer; a device may optionally have an assigned repair ticket, while every repair ticket must have an associated device; etc.).

- A **weak entity** exists:
  - Recall that a **REPAIR_TICKET** may be worked on by multiple **TECHNICIAN**s and that each **TECHNICIAN** works on many **REPAIR_TICKETs**. In other words, this is a m:m relationship that uses the intersection entity **TECH_TICKET**.
  - So far, so good. However, we have learned that **TECH_TICKET** does not have its own unique identifier, as we originally assumed. Rather, each **TECH_TICKET** is uniquely identified using a composition of identifiers from **REPAIR_TICKET** (**ticket_id**) and **TECHNICIAN** (**tech_id**). This requires removing the **tt_id** attribute from **TECH_TICKET** and adding unique identifier (UID) bars on the m-side from both **REPAIR_TICKET** and **TECHNICIAN**.
- A **recursive relationship** exists:
  - A **TECHNICIAN** may optionally supervise multiple other technicians, but each technician has zero or one supervisor.

A revised interpretation based on this additional information is shown below:

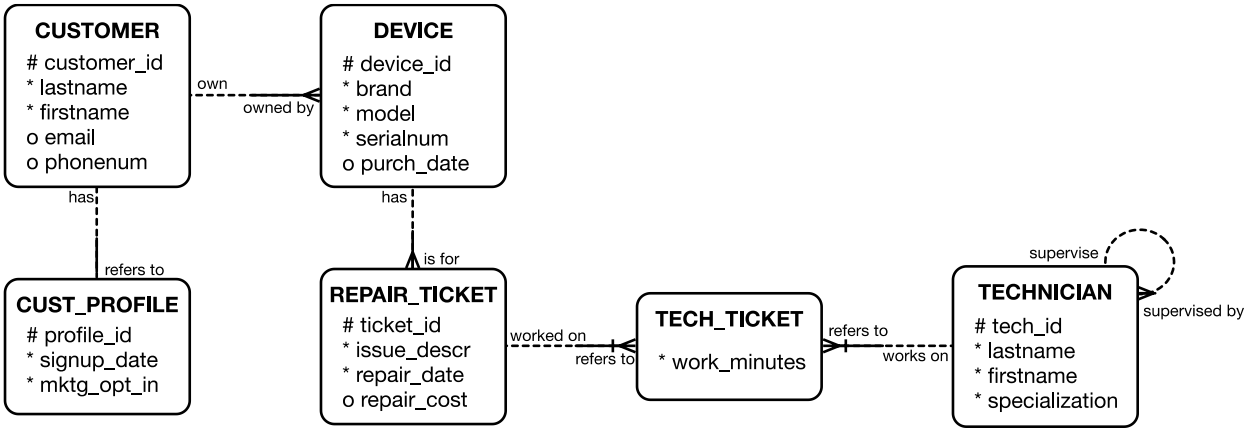


EXHIBIT 1: ER DIAGRAMMING BEST PRACTICES

1. **Consistency.** Maintaining consistency in naming conventions, symbols, and notation throughout all stages of ER modelling is crucial. This uniformity ensures clarity, reduces confusion, and makes diagrams more intuitive and easier to understand. *Consider establishing a style guide or template for ER diagrams within your organization to ensure consistency across different projects and teams.*

2. **Iteration.** ER modelling should be viewed as an iterative process. Regular reviews and updates to the ER diagrams are essential as business requirements evolve and more information becomes available. This iterative refinement helps maintain the accuracy and relevance of the model. *Schedule periodic reviews and feedback sessions with key stakeholders to systematically incorporate changes and improvements.*

3. **Stakeholders.** Involving stakeholders throughout the ER modelling process is critical. Their input ensures that the model accurately reflects business needs and realities. This collaborative approach helps in identifying and addressing any discrepancies or misunderstandings early in the process. *Facilitate workshops or collaborative sessions with stakeholders to gather requirements, validate the model, and educate stakeholders on the importance and basics of ER modelling.*

4. **Validation.** Regularly validate the ER diagram against business requirements and rules. This ensures that the model remains aligned with the business objectives and can effectively support the necessary data processes and workflows. *Develop a checklist of business rules and requirements for ER diagramming, and use it during validation sessions to ensure comprehensive coverage.*

5. **Readability.** The primary goal of an ER diagram is to capture and codify information clearly and efficiently. Avoid cluttering the diagram with excessive details that can be represented in other ways, such as documentation. A clean, well-organized diagram is more effective as a communication tool. *Employ tools such as colour-coding and layering to differentiate between various aspects of the model without overwhelming the viewer.*

6. **Documentation.** Supplement ER diagrams with thorough documentation, including detailed descriptions of entities, attributes, relationships, and any assumptions made during the modelling process. *Documentation provides context and clarity, especially for those who are unfamiliar with the diagram's intricacies.*

7. **Scalability.** Design ER diagrams with scalability in mind. As the business grows, the data model should be able to accommodate new requirements without major overhauls. *A scalable design ensures the long-term viability and adaptability of the data model.*

8. **Training.** Provide training sessions for team members on ER modelling best practices, tools, and techniques. *A well-informed team can produce higher-quality models and effectively collaborate on iterative improvements.*

Source: Case author.

EXHIBIT 2: ER DIAGRAM SOFTWARE TOOLS

Many different graphical tools may be used to create data models. However, in the early stages of model design, sticking with pencil and paper is recommended. This has several benefits: manual graphics offer flexibility, allowing for quick modifications and spontaneous brainstorming, which can stimulate creativity and the exploration of different ideas and structures. It also helps to focus on the conceptual aspects of the design without getting bogged down by technical details or software-specific constraints. Starting with pencil and paper can also help prevent over-engineering, ensuring simplicity and clarity in the foundational aspects of the model.

Once the basic structure is well-conceived, these sketches can be translated into more precise diagrams using graphical tools, ensuring a solid and accurately documented final data model. Numerous commercial and free software tools are available to do this, with varying complexity and features. Examples of such tools include the following:

- **Microsoft Visio**: Offers a wide range of templates and shapes specifically designed for ER diagramming. It is a comprehensive tool favoured for its flexibility and integration with other Microsoft Office products. Platforms: PC.
- **OmniGraffle**: A powerful diagramming tool for Mac users that provides robust features for creating detailed and precise ER diagrams. It is known for its intuitive interface and versatility in handling various types of diagrams. Platforms: Mac.
- **Lucidchart**: A cloud-based solution that supports real-time collaboration and easy sharing. It is known for its user-friendly interface and robust features, which are suitable for both simple and complex diagrams. Platforms: Web (PC and Mac).
- **SmartDraw**: An online diagramming tool that offers extensive templates and intuitive design capabilities for creating professional ER diagrams. It is suitable for both novice and advanced users. **Platforms**: Web (PC and Mac).
- **draw.io (diagrams.net)**: A versatile and user-friendly online tool that is free to use. It supports a wide variety of diagram types, making it a great choice for quick and easy ER diagram creation. Platforms: Web (PC and Mac).

Source: Case author.